

Ankura: корректное нейросимволическое рассуждение с проверяемым выводом

Владимир Лапшин*

Аннотация

Мы представляем Ankura — нейросимволическую систему рассуждений, не зависящую от предметной области. В её основе лежит принцип разделения ролей: языковая модель *предлагает* формализацию, а детерминированный символический решатель её *решает*. Система охватывает несколько разрешимых формализмов — от определённых хорновских клауз и стратифицированного отрицания до позитивной логики первого порядка с дизъюнкцией и кванторами, конечно-доменных ограничений, точной арифметики и умолчаний со специфичностью. Для каждого формализма определена корректная решающая процедура и объявлен фрагмент. Любой ответ сопровождается автоматически проверяемой историей вывода; ни одно утверждение не попадает в теорию без дословной цитаты или явной пометки гипотезы. Мы приводим демонстрационную оценку на стандартных коллекциях: на охваченных срезах система показывает высокую точность, не допуская ни одного уверенно неверного вывода, и честно воздерживается (*out_of_fragment*) там, где реализованная логика не может выразить задачу. Мы также сравниваем систему с двумя открытыми нейросимволическими системами — Logic-LM и LINC — на одинаковых входах и одной и той же модели и показываем, что при малой и недорогой модели связывающим ограничением оказывается интерфейс перевода, а не рассуждение. Мы не претендуем на передовой уровень: оценка опирается на зафиксированные подвыборки, а не на полные тестовые наборы, и все возникающие статистические и методические ограничения мы проговариваем явно.

1. Введение

Большие языковые модели хорошо читают текст и переводят его на точный формальный язык. Плохо они делают то, что удаётся символическим процедурам: выводят следствия *корректно*, точно отслеживают, какие посылки были использованы, и отказываются угадывать. Свободная цепочка рассуждений (*chain-of-thought*) звучит правдоподобно, но не отличает правильный шаг от просто убедительного: у неё нет ни свидетельства корректности, ни истории, по которой можно было бы восстановить, на чём держится ответ. Именно поэтому проверяемость оказывается центральной проблемой рассуждений языковых моделей.

Ankura [1] даёт каждой стороне ту работу, в которой она сильна. Языковая модель читает текст и выполняет две вещи: разбирает задачу на *структуру* (объекты, факты, правила, цитаты) и, пока ответ ещё открыт, предлагает отдельные шаги. Символический решатель отвечает за *формальную* сторону: превращает структуру в теорию, замыкает её по правилам вывода, сопоставляет с вопросом и удостоверяет то, что действительно следует.

Центральный принцип построения формулируется коротко: *языковая модель предлагает, решатель решает*. Модель никогда не владеет истиной. Каждое её утверждение детерми-

*merfill@yandex.ru

нированно классифицируется как *derivable, cited* (подкреплено дословной цитатой), *hypothesis* (знание, отсутствующее в тексте, с пометкой) или *rejected*. Обоснованность ответа никогда не превышает обоснованности его посылок: вывод из цитат — это *proven*, вывод, опирающийся на допущения, — *proven_under(H)*.

Это разделение не является инженерным компромиссом. Оно отражает различие *неформализуемого источника* (*doxa*) и формального *logos*: языковая модель здесь — носитель *doxa*, к которому *logos* обращается за предложениями, но не за истиной, а протокол предложений, детерминированная классификация, журнал и проверки привязки к тексту образуют интерфейс между ними [2]. Полное изложение операторного семейства — предмет отдельной работы; здесь мы сосредоточены на реализующей его системе и её эмпирической проверке.

Вклад.

- Архитектура «предлагает / решает» с двумя отдельными участниками и автоматически проверяемой историей вывода.
- Протокол предложений: каждое утверждение классифицируется детерминированно, а обоснованность ответа (*proven, proven_under(H), not_proven*) никогда не превышает обоснованности его посылок.
- Семейство разрешимых формализмов в рамках *протокола объявленного фрагмента* с честным воздержанием *out_of_fragment* вместо молчаливого перехода к более слабой логике.
- Методика поэтапных проверок *составных* эталонных коллекций, разделяющая потолок охвата (формализм) и потолок извлечения (модель) с помощью режима эталонного входа.
- Сравнение на общем входе с двумя открытыми системами вида «переводчик + решатель» на тех же строках и той же модели, разделяющее интерфейс перевода и символическое рассуждение и оценивающее эффект режима модели.
- Демонстрационный прототип на стандартных коллекциях с измеримым инвариантом корректности — отсутствием уверенно неверных выводов.

Область (явно). Это отчёт о демонстрационном прототипе, а не заявка на передовой уровень. Мы оцениваем *зафиксированные подвыборки* стандартных коллекций, объясняем бюджет, не позволяющий проходить полные тестовые наборы, и приводим доверительные интервалы. Полный охват коллекции недостижим по построению для наборов со смешанными конструкциями, таких как FOLIO (раздел 6).

Структура. Раздел 2 описывает архитектуру и протокол предложений; раздел 3 приводит примеры по стадиям; раздел 4 перечисляет формализмы и решающие процедуры; раздел 5 излагает методику оценки; раздел 6 приводит результаты; раздел 7 сравнивает систему с Logic-LM и LINC на одинаковых входах; раздел 8 — ограничения; раздел 9 — связанные работы.

2. Обзор системы

Ankura — не монолитный решатель, а система, которая координирует формализации и решающие процедуры. Для каждой задачи модель порождает формализацию в некото-

рой логике, а корректная процедура для этой логики её решает. Разные классы задач требуют разных процедур, каждая со своим внутренним представлением; какие именно логики реализованы, описано в разделе 4.

2.1. Четыре фазы

Задача проходит четыре фазы. Только одна из них — вывод — принадлежит решателю; две другие обращаются к модели.

- **Фаза 0 — декомпозиция.** Текст разделяется на описательную и вопросительную части, и с этого момента они трактуются как разные объекты. Далее следуют два обращения к языковой модели в фиксированном порядке: сначала структурная декомпозиция описательной части (объекты, факты, условные утверждения, альтернативы и группировки, отрицания и модальности, короткая дословная цитата на каждый элемент), затем выражение вопроса над уже установленным каноническим словарём теории. Между обращениями и после них детерминированный построитель разворачивает и дополняет структуру до *теории* и *запроса*. Действует строгое правило: *вопрос никогда не утверждается как факт*.
- **Фаза 1 — решение.** Символический решатель выводит все следствия теории до неподвижной точки и проверяет, следует ли цель. Каждый выведенный факт несёт историю: какие факты и какое правило его породили. Решатель не добавляет знания от себя — он корректен и полон *относительно* полученной формализации.
- **Фаза 2 — управляемый цикл.** Если построенная теория ещё не решает вопрос, система не останавливается на голом «неизвестно»: она сообщает, где именно возникла трудность, и модель получает ограниченную возможность помочь, предлагая по одному шагу за итерацию (раздел 2.3).
- **Фаза 3 — объяснение.** Готовая трасса вывода собирается *автоматически* из истории вывода, а не моделью. Модель может позже пересказать готовую трассу связным текстом, но не вправе вводить новые факты.

Ключевая граница проходит между фазами 0 и 1: всё, что формулирует модель, обязано либо опираться на дословную цитату, либо быть явно помеченным допущением. Ничто не входит в теорию без такого основания.

2.2. Протокол предложений

Каждое утверждение, которое модель хочет добавить, классифицируется детерминированно. Классификация не является субъективным суждением: она зависит только от того, выводится ли утверждение уже сейчас, подкреплено ли оно текстом или является допущением.

Категория	Условие	Действие решателя
derivable	уже выводится из текущей теории	принимает только как изложение
cited	новое утверждение с дословной цитатой из текста	добавляет в аксиомы
hypothesis	знание, отсутствующее в тексте	заносят в журнал с пометкой <i>H</i>
rejected	не проходит проверку схемы, без-опасности или цитаты	отклоняет с кодом причины

Именно различие `cited` и `hypothesis` делает ответы честными. Вывод, использующий только процитированные факты, — это `proven`. Вывод, опирающийся на допущения, — `proven_under(H)`, и ответ называет эти допущения. Если ничего не установлено, ответ — `not_proven`. Поэтому обоснованность ответа никогда не превышает обоснованности его посылок, а допущение не может выдавать себя за факт.

2.3. Управляемый цикл и журнал

Внутренний цикл (фаза 2) полностью детерминирован; обращением к модели является только *предложение*. Цикл чередует подсказку, свободное предложение и детерминированную классификацию и принимает не более одного предложения за итерацию — так каждая итерация оставляет проверяемый след. Итерация устроена следующим образом:

1. **Проверка.** Решатель замыкает текущую теорию и проверяет цель, порождая вердикт и структурированные пробелы (например, `target_unmatched:`, когда предикат цели нигде не встречается).
2. **Проверка завершения.** Если вердикт уже решающий — `supported`, `refuted`, `contradiction` или `insufficient` — цикл останавливается.
3. **Подсказка.** Модель получает структуру задачи, допустимые предикаты и объекты теории, сводку выведенного, текущие пробелы, цель и виды допустимого предложения.
4. **Предложение.** Модель возвращает краткое изложение и ровно одно типизированное предложение: переформулировать вопрос в словаре теории, добавить правило, утвердить цитируемый факт или выбрать следующую подцель.
5. **Классификация и применение.** Решатель проверяет предложение на схему, правила безопасности и текст, относит его к одной из четырёх категорий выше и применяет либо отклоняет. Применение монотонно: теория только растёт.
6. **Запись.** Итерация заносится в журнал, внутренний цикл перезапускается.

Цикл останавливается на решающем вердикте, на `no_progress` (две подряд итерации без изменений), на исчерпании бюджета итераций или когда допущения запрещены. Ни один из этих исходов не является нарушением корректности: каждый — честная точка остановки.

Монотонная база, меняющийся ответ. База — аксиомы, правила, гипотезы — только растёт: посылки не удаляются, цель не ослабляется. *Ответ* тем не менее может меняться по мере поступления новой информации. Такие изменения не скрываются: каждое записывается как `Revision` с указанием итерации-источника и причастных допущений, и ревизии сохраняются вместе с объяснением. Так система остаётся проверяемой при немонотонном рассуждении, не изменяя при этом базу.

2.4. Модель данных

Всё, о чём рассуждает система, собирается из нескольких простых элементов. *Объект* (`Object`) — каноническая именованная сущность. *Отношение* (`Morphism`) — один предикат, применённый к субъекту и объекту, с модальностью и признаком отрицания; отрицание — тот же предикат, помеченный ложным, а не отдельное имя. *Правило* (`Rule`) — условное утверждение: конъюнкция условий влечёт следствие, оно помечено как импликация или исключение и несёт источник (цитату или идентификатор гипотезы). *Теория*

(Theory) собирает объекты, утверждённые отношения и правила, а *запрос* (Query) хранит условия, которые утверждает сам вопрос (Γ), цель (ϕ) и тип ответа. Внутри решателя *факт* (Fact) — это заземлённый атом вместе с историей вывода.

2.5. Принципиальное ограничение: детерминированный код не разбирает язык

Детерминированный код никогда не интерпретирует естественный язык. Нормализация идентификаторов и закрытых перечислений, которые модель уже извлекла (идентификаторы предикатов и объектов, модальность, `relation_kind`, ключи полей), допустима: она действует на структурированный результат, а не на исходный текст. Эвристики по исходному тексту, опирающиеся на слова, морфологию, характерные обороты или составленные вручную списки терминов, недопустимы: они не обобщаются между предметными областями, а семантика задаётся инструкцией извлечения. Единственные операции над исходным текстом — структурные проверки: точное вхождение цитаты, покрытие и длина.

3. Примеры по стадиям

Ниже приведены примеры для каждой стадии — от чистой дедукции и абдукции до умолчаний, отрицания, дизъюнкции, составного случая FOLIO, конечно-доменных ограничений и точной арифметики. Они показывают, как работают протокол предложений, обоснованность ответа и выбор решающей процедуры на конкретных входах.

3.1. Чистая дедукция

Текст: «*It is raining. If it is raining, the ground is wet. Is the ground wet?*»

Фаза 0 разделяет текст на описательную и вопросительную части. Описательная часть даёт один факт и одно правило, вопрос становится целью:

Элемент	Содержимое	Источник
Факт	<code>raining()</code>	<code>cited</code> (« <i>It is raining.</i> »)
Правило R_1	<code>raining() $\Rightarrow$ is_wet(ground)</code>	<code>cited</code> (« <i>If it is raining, the ground is wet.</i> »)
Цель ϕ	<code>is_wet(ground)</code> , тип <code>yes_no</code>	вопрос (не факт)

Фаза 1 замыкает теорию: правило R_1 срабатывает на факте и выводит `is_wet(ground)`, сохраняя в истории посылку и сработавшее правило. Проверка даёт вердикт `supported`, ответ `yes` и обоснованность `proven`; цикл не выполняет ни одного обращения к модели. Фаза 3 собирает трассу автоматически:

1. `raining()` — аксиома, `cited`;
2. `is_wet(ground)` — правило R_1 , из шага 1.

3.2. Абдукция и обоснованность ответа

Текст: «*The vehicle has an engine, four wheels, 150 horsepower and four doors. What kind of vehicle is it?*»

Фаза 0 извлекает только наблюдаемые факты — `has_engine`, `wheel_count`, `power`, `door_count`. Правил классификации в тексте нет, поэтому их нет и в теории. Цель

открыта: `is_a(X, ?c)`. Фаза 1 не выводит для этого предиката ничего и сообщает пробел `target_unmatched: is_a`, после чего запускается управляемый цикл.

На первой итерации модель предлагает правила классификации, например: «двигатель $\Rightarrow$ моторное средство», «моторное средство и четыре колеса $\Rightarrow$ автомобиль». Цитат под ними нет, поэтому классификация относит их к `hypothesis`, и правила заносятся в журнал с пометками $H_1, H_2, \dots$ (раздел 2.3). Теория монотонно растёт, замыкание выводит `is_a(X, car)` и `is_a(X, sedan)`, а переменная цели связывается по цепочке `is_a`, а не угадыванием. Проверка даёт `supported`, но вывод опирается на допущения, поэтому обоснованность ответа — `proven_under(H_1, H_2, \dots)`, и ответ называет эти допущения поимённо.

Если запустить ту же задачу с `ANKYRA_ALLOW_HYPOTHESES=false`, те же правила отклоняются, и система честно отвечает `not_proven`. Разница между `proven` и `proven_under` в этих двух примерах не косметическая: она в точности отражает, на чём держится ответ.

3.3. Умолчания и специфичность

Текст: «*Birds can fly. Penguins are birds, but penguins cannot fly. Tweety is a penguin. Can Tweety fly?*»

Фаза 0 даёт факты `is_a(tweety, penguin)` и `is_a(penguin, bird)`, а также два правила-умолчания: `is_a(?x, bird)  $\Rightarrow$  fly(?x)` и `is_a(?x, penguin)  $\Rightarrow$   $\neg$ fly(?x)`. При обычном монотонном замыкании сработали бы оба правила и получилось бы противоречие. При включённом слое умолчаний правило про пингвинов оказывается строго специфичнее — оно опирается на ребро `is_a(penguin, bird)` — и побеждает: вердикт `refuted`, ответ «нет», признак `defeasible=true`.

Симметричный пример — «птицы не летают, пингвины летают» — даёт `supported` и ответ «да»: специфичность не зависит от полярности. Если же взять классический пример «Никсон — квакер и республиканец» с правилами `quaker  $\Rightarrow$  pacifist` и `republican  $\Rightarrow$   $\neg$ pacifist`, специфичность не определена — оба правила одинаково конкретны, — и система честно отвечает `unsupported` с пробелом `undecided_conflict`, а не угадывает победителя. Источник: `evals/defeasible_synthetic`, 8/8.

3.4. Отрицание и несовместность (L1)

Текст: «*No real number is imaginary. a is a real number. Is a imaginary?*»

Утверждение «ни одно действительное число не мнимо» формализуется не общим правилом, а ограничением несовместности `disjoint(real_number, imaginary)`. Вместе с фактом `is_a(a, real_number)` оно даёт `$\neg$ is_a(a, imaginary)` и, следовательно, опровергает цель: вердикт `refuted`, ответ «нет». Существенно, что опровержение опирается на явную несовместность, а не на молчаливое закрытие мира: при открытом мире система не объявляет цель ложной лишь потому, что её не удалось вывести. Источник: `evals/l1_synthetic`, идентификатор `disjoint-refute-01`; проверка 40/40.

3.5. Дизъюнкция и разбор случаев (L2)

Теория: факт `is_a(rex, prim)`; правило `is_a(?x, prim)  $\Rightarrow$  is_a(?x, a)  $\vee$  is_a(?x, b)`; правила `is_a(?x, a)  $\Rightarrow$  is_a(?x, c)` и `is_a(?x, b)  $\Rightarrow$  is_a(?x, c)`; цель — `is_a(rex, c)`. Хорновская процедура не может обработать дизъюнктивную голову, поэтому включается клаузная процедура L2: она разбирает случаи по двум

дизъюнктам и в каждом выводит с, после чего вердикт — supported. С выключенным L2 тот же вход даёт честный out_of_fragment, а не молчаливый переход к более слабой логике. Источник: evals/l2_synthetic, идентификатор head-case-split-01; проверка 65/65.

3.6. Составной случай FOLIO

Эталонная разметка FOLIO смешивает конструкции разных стадий, поэтому коллекция оценивается по внутрифрагментным срезам. В строке folio-validation-0010 («дикая индейка») посылки содержат дизъюнкцию шести подвидов и факт WildTurkey(tom). Условные отрицания вместе с этим фактом дают ¬Eastern(tom), ¬Osceola(tom) и отрицание дизъюнкции Goulds, Merriams и Riogrande; следовательно, цель Eastern(tom), помеченная False, опровергается. Клаузальная процедура даёт refuted («нет»), что совпадает с меткой.

Строка folio-validation-0027 («Марвин») устроена иначе: цель помечена False, но размеченные посылки не влекут ни саму цель, ни её отрицание, и система честно воздерживается (unknown), а не выдаёт неподтверждённое «нет». На срезе отрицания (13 строк) режим эталонного входа даёт 12/13 при нуле уверенно неверных выводов; на срезе L2 уровня а (45 строк) охват составляет 44/45, а режим эталонного входа — 42/44. Источники: evals/data/folio_l2_tier_a.jsonl, folio_negation_tier_a.jsonl.

3.7. Конечно-доменные ограничения (L3)

Задача AR-LSAT: пять инструментальных пьес идут на компакт-диске в некотором порядке; заданы условия порядка, соседства и группировки, а вопрос спрашивает, какая из предложенных последовательностей допустима. Формализация — конечно-доменная: область позиций 0..4, пять переменных с ограничением all_different, порядковые и составные ограничения. Вопрос вида not_violate проверяется перебором, а вопросы «обязательно» и «может быть» решаются как истинность во всех или хотя бы в одном допустимом размещении. На закодированных вручную реальных играх режим эталонного входа даёт 27/27, в том числе 5/5 для вопросов not_violate; реальный запуск на разбиении eval — 23/30. Источники: evals/data/ar_lsats_dev.jsonl, evals/data/ar_lsats_gold.jsonl.

3.8. Точная арифметика (L4)

Задача GSM8K: «Генри сделал две остановки в поездке на 60 миль; первая — через 20 миль, вторая — за 15 миль до конца. Сколько миль он проехал между остановками?» Фазза 0 строит ориентированный граф величин: total=60, first=20, before_end=15, а искомая величина between задаётся уравнением. Линейная система решается в точных рациональных числах: $60 - 20 - 15 = 25$. Арифметическая ошибка здесь невозможна по построению: неверное число может быть только ошибкой моделирования, но не счёта. Источник: evals/data/gsm8k_gold.jsonl, идентификатор gsm8k-gold-0028; режим эталонного входа 8/8.

4. Формализмы и решающие процедуры

Ankura — не «решатель хорновских клауз», а система, координирующая формализации. Конечная цель — охватить наибольший практически важный класс рассуждений, для которых существует разрешимая процедура проверки: модель порождает формализацию

в логике L , а корректная процедура для L её решает. Охват поэтому измеряется в именованных формализмах, а не в неопределённом «рассуждении».

4.1. Протокол стадии

Каждая стадия описывается одинаковым набором из шести пунктов: *формализм* L и семантика ответа; *решающая процедура*, корректная для L (и, где заявлено, полная); *изменение решателя*; *изменение извлечения* — то, что фазы 0 и 2 должны уметь выразить, чтобы модель могла предлагать в L ; *этапная проверка на эталонной коллекции* с малой зафиксированной выборкой и жёсткими инвариантами (отсутствие уверенно неверных выводов, каждый определённый ответ proven, точность выше порога, каждый остаток отнесён к категории); и *переключатель*, выключенный по умолчанию, чтобы семантика эталонной коллекции не проникала в решатель.

Стадия	Формализм	Процедура
L0	определённые хорновские клаузы, <code>is_a</code> , отрицание как комплементарные пары (открытый мир)	полунаивный прямой вывод до неподвижной точки
L1	стратифицированное отрицание / отрицание как неудача (NAF), объявленный закрытый мир	стратифицированное замыкание, ограничения несовместности
L2	позитивная логика первого порядка: дизъюнкция, $\exists/\forall$, доказательство разбором случаев	ограниченная резолюция (заземление по конечной области)
L3	конечно-доменные ограничения (CSP/SAT)	поиск по конечным областям значений
L4	арифметические термы и уравнения	точное рациональное вычисление / линейное исключение
D	умолчания со специфичностью через <code>is_a</code>	слой умолчаний (специфичность над строгим ядром)

L3 и L4 — отдельные решатели в рамках той же системы, со своим внутренним представлением: конечно-доменные ограничения — это не логическая выводимость, а удовлетворение ограничений, а арифметическая задача даёт число, а не следствие.

4.2. Объявленный фрагмент

Поскольку процедуры разные, система должна решить, какая из них запускается, — и сделать это, никогда не угадывая. Ответ даёт *протокол объявленного фрагмента*. Система анализирует *построенные* теорию и запрос и выводит набор требуемых признаков: есть ли дизъюнктивная голова, квантор существования, составная цель, отрицание, встроенное сравнение, равенство. Это и есть *фрагмент* — объективное свойство формализации, а не намерение. Отдельно запуск *объявляет* свою семантику: какие процедуры включены (переключатели `ANKYRA_*`) и читается ли запрос при открытом или закрытом мире. Решение о выборе процедуры сводит эти два начала и проверяет их друг против друга.

Если фрагмент укладывается во включённые возможности, выбирается соответствующая процедура. Если нет — структуре нужна логика, которую запуск не включил, или комбинация, которую выбранная процедура не может соблюсти, — результатом становится именованный пробел `out_of_fragment`, а не молчаливый переход к более слабой

процедуре.

Точкой расширения служит протокол *Inference*: семантика (выводимость, форма вердикта, замыкание, против которого классифицируется предложение) становится сменной, а выбор процедуры и механизмы остаются на месте. Сегодня у него две реализации — хорновская и клаузная; L3 и L4 подключаются как отдельные решатели.

4.3. Что общего у решателей

Каждый решатель корректен *относительно* переданной ему формализации и честно поддерживается вместо угадывания. Символические компоненты полны относительно формализации (прямой вывод достигает неподвижной точки), но полнота *перевода* не заявляется: модель всегда может упустить посылку в тексте. Поэтому различие между неверным и недостающим ответом сохраняется всюду.

5. Методика оценки

Метрики. Для каждой коллекции мы приводим *точность по классу* (трёхклассовую или с выбором варианта) с доверительными интервалами Уилсона (95%). Этапная проверка также требует *инварианта корректности*: каждый определённый ответ *proven* и отсутствие уверенно неверных выводов.

Два потолка. Для составной эталонной коллекции мы разделяем *охват* (долю эталонных формул, которые реализованная логика может выразить и решить; остаток — честный *out_of_fragment*) и *извлечение* (долю охваченных строк, которые языковая модель переводит верно). Режим *эталонного входа* подаёт решателю размеченную формализацию вместо извлечённой моделью, отделяя точность метода от точности перевода.

Бюджет и воспроизводимость. Бесплатного доступа к языковой модели нет; запуски с обращением к ней платные. Поэтому каждая коллекция входит небольшой зафиксированной подвыборкой, проходит один раз и повторяется только при расхождении. Все проверки без участия языковой модели (синтетические наборы, диагностика с эталонным входом) воспроизводимы из одного репозитория.

Лестница проверок. Результаты добываются на лестнице из трёх ступеней. Первая — синтетические наборы без участия языковой модели, напрямую использующие семантику каждого формализма, без извлечения и без изменчивости провайдера доступа к LLM: это чистый сигнал решателя. Вторая — простая по поверхности реальная коллекция (например, ProntoQA-OOD для L2), добавляющая реальный текст поверх чистой нотации. Третья — намеренно тяжёлый по извлечению срез реальной коллекции (FOLIO). Поперёк лестницы режим *эталонного входа*, то есть подача решателю написанных вручную формул вместо извлечённых, разделяет два разных предела: *охват* (сколько задач реализованная логика вообще может выразить; выход за него — честный *out_of_fragment*, снижающий полноту, но не корректность) и *извлечение* (сколько охваченных задач модель переводит верно). Разделение этих двух и показывает, откуда должно прийти улучшение — из решателя или из интерфейса.

6. Результаты

Мы сообщаем два результата: точность по классу на охваченных срезах и инвариант корректности. Сначала идут детерминированные проверки без участия языковой модели, затем реальные запуски на внешних коллекциях. Ни на одном отчитанном срезе нет ни одного уверенно неверного вывода, и каждый определённый ответ помечен `proven`.

6.1. Синтетические проверки без участия языковой модели

Эти проверки детерминированы и покрывают каждую конструкцию каждого формализма (включая отрицательные контроли); языковая модель в цикле не участвует.

Проверка	Результат	Уверенно неверных выводов
Выбор процедуры (<code>evals.routing_synthetic</code>)	19/19	0
L1 отрицание (<code>evals.l1_synthetic</code>)	40/40	0
L2 клаузная + равенство (<code>evals.l2_synthetic</code>)	65/65	0
L3 конечно-доменный CSP (<code>evals.l3_synthetic</code>)	31/31	0
L4 арифметика (<code>evals.l4_synthetic</code>)	37/37	0
Умолчания (<code>evals.defeasible_synthetic</code>)	8/8	0

6.2. Внешние коллекции

Таблица 1 сводит реальные и зафиксированные результаты. Столбец уверенно неверных выводов — инвариант корректности (0 в каждой строке).

Стадия	Коллекция / срез	Результат	95% ДИ
L0	ProofWriter, уровень D (открытый мир)	297/299 (99,3%)	[97,6; 99,8]
L1	ProntoQA, уровень a	48/48 (100%)	[92,6; 100]
L1	ProntoQA, уровень b	160/160 (100%)	[97,7; 100]
L2	ProntoQA-OOD, уровень a	41/42 (97,6%)	[87,7; 99,6]
L2	FOLIO, срез отрицания (текстовый вход)	11/13 (84,6%)	[57,8; 95,7]
L2	FOLIO L2, уровень a (текстовый вход, охваченные)	29/44 (65,9%)	[51,1; 78,1]
L3	AR-LSAT, eval	23/30 (76,7%)	[59,1; 88,2]
L4	GSM8K, eval	38/40 (95,0%)	[83,5; 98,6]

Таблица 1: Результаты на зафиксированных подвыборках. Знаменатели — малые выборки, а не полные тестовые наборы; см. раздел 8.

ProofWriter (L0). Это базовая проверка определённого хорновского слоя. На малом строгом срезе уровня A — 45/45; на расширенном уровне D — 297/299 (99,3%) по оцениваемым записям, ни одного уверенно неверного вывода и каждый определённый ответ `proven`. Одна из 300 отобранных записей исключена из знаменателя: вопрос не удалось извлечь (изменчивость поставщика), а адресный повторный запуск ответил на неё `proven`. Два оставшихся промаха — честные ответы `unknown` при ошибке извлечения, а не неверные заключения. Полная коллекция (6 368 теорий, порядка 11 часов) не запускалась по бюджетным причинам.

ProntoQA (L1) и ProntoQA-OOD (L2). Это проверки на реальном тексте поверх чистой нотации. ProntoQA проверяет явное отрицание: уровень a — 48/48, уровень b — 160/160,

все ответы proven; односторонняя 95%-я граница Клоппера—Пирсона по точности на задачу для уровня b составляет 98,1%. ProntoQA-OOD проверяет композиционные рассуждения (дизъюнкция, разбор случаев, кванторы): уровень a — 41/42 (97,6%), ни одного уверенно неверного вывода. После этих запусков коллекция ProntoQA считается закрытой.

FOLIO (L1/L2, составной случай). Разметка FOLIO в логике первого порядка охватывает несколько формализмов сразу, поэтому она оценивается по внутрифрагментным срезам, а не единым баллом:

- срез отрицания (13 строк) оказался *ограниченным фрагментом*: на хорновском пути режимы эталонного и текстового входа совпадают (7/13), а остаток составляют сведения к противоречию и контрапозиция; поэтому срез оценивается клаузуальной процедурой L2: эталонный вход — 12/13, текстовый вход — 11/13, ни одного уверенно неверного вывода. Четыре ответа Uncertain остаются честными unknown; закрытый мир глобально не применяется, иначе он неверно опроверг бы их;
- уровень a формализма L2 (45 строк): охват — 44/45 (одна повреждённая разметка), то есть реализованная логика выражает почти весь срез; на охваченных строках режим эталонного входа даёт 42/44 против 29/44 при текстовом входе.

Разрыв между режимом эталонного входа (метод) и текстовым входом (извлечение) локализует оставшуюся стоимость в извлечении, а не в решателе: потолок охвата на зафиксированном 45-строчном срезе фактически достигнут. По всему 204-строчному проверочному разбиению два зафиксированных фрагмента проверок покрывают $13 + 93 = 106$ строк (нижняя оценка выразимой доли); из 93 строк фрагмента L2 зафиксированы и прогнаны только 45, а 48 оставлены непрогнанными по бюджетным причинам. Оставшиеся строки — не более 98 — содержат конструкции вне любой реализованной стадии (исключающее или, эквивалентность, кванторы по нескольким переменным, равенство, функциональные термы, схемы аксиом) и отчитываются как `out_of_fragment`.

AR-LSAT (L3). Конечно-доменные ограничения проверяются отдельным решателем. На закодированных вручную реальных игровых задачах режим эталонного входа даёт 27/27; реальный запуск на разбиении eval — 23/30 (76,7%) без единого `grounded_mismatch`. Семь промахов — честные воздержания, а не уверенно неверные ответы.

GSM8K (L4). Точная рациональная арифметика делает арифметические ошибки невозможными по построению; неверное число здесь всегда ошибка *моделирования*, а не счёта. Синтетические проверки — 37/37, закодированный вручную эталон — 8/8, реальный запуск: разбиение dev — 12/12, разбиение eval — 38/40. Две неверные строки разбиения eval — размеченные ошибки эталона: одна ошибка в наборе данных, одна неоднозначность.

7. Сравнение с Logic-LM и LINC

Формализмы и проверки выше — вклад этой работы. Естественно спросить, как те же задачи решают две наиболее близкие открытые нейросимволические системы — Logic-LM [3] и LINC [4]. Обе устроены по схеме «переводчик + решатель»: языковая модель строит формализацию, символический решатель её решает. Вместо качественного противопоставления (как в разделе 9) мы приводим сравнение *на общем входе* — одни и те же строки задач и одна и та же языковая модель для всех трёх систем, — а затем разбираем, что именно их разделяет.

7.1. Протокол

Входы и модель. Каждая система получает ровно те же строки, что и в примерах для Ankyra выше, и ту же модель, которую решатель использует во время запуска: малую и недорогую инструктивную модель (DeepSeek V4 Flash) за OpenAI-совместимой конечной точкой. Внешние системы не переписывались: их собственные подсказки, символические решатели и процедуры подсчёта оценок остаются неизменными, их лишь оборачивает тонкий адаптер. Адаптер необходим так как обе библиотеки рассчитаны на устаревший клиент `openai<1`; адаптер перенаправляет этот клиент на провайдера решателя, подменяет модель и отключает явную цепочку рассуждений — так же, как это делает сам решатель. Вызовы при температуре 0 кэшируются, поэтому запуски воспроизводимы без дополнительных затрат.

Декодирование. Каждая система запускается в собственном протоколе декодирования, потому что именно к нему относятся её опубликованные числа. Под сэмплированием здесь понимается многократный запуск LLM на одном и том же входе с получением N вариантов ответа. Оно используется для оценки устойчивости генерации и для повышения шанса, что хотя бы один вариант пройдет разбор.

Система	Декодирование	Почему
Ankyra	температура 0,1, один сэмпл, структурированное (tool/JSON) извлечение	так устроен решатель; он не сэмплирует
Logic-LM	температура 0, один сэмпл, плюс её self-refinement	её опубликованный конвейер
LINC	8 примеров, 10 сэмплов, температура 0,8, голосование большинством	её опубликованный конвейер

Прогон с согласованным декодированием (все три системы жадные, один сэмпл) приведён в разделе 7.6, чтобы исключить влияние самого протокола на результат.

Метрики. *Точность решателя* — доля строк, на которые дан верный ответ. При этом и неудачный запуск программы, и отказ от ответа считаются ошибкой. Это самая строгая из наших метрик, и всюду далее мы используем именно её. *Охват* — доля строк, для которых программу удалось запустить. *Точность среди запустившихся* — точность только по тем строкам, где программа отработала. Уверенно неверные ответы — запустившиеся программы, давшие неверный ответ: внешние системы выдают такой ответ как окончательный, тогда как Ankyra корректна относительно своей формализации и подобных ответов не порождает.

Область. Это не воспроизведение и проверка результатов двух статей: в них использовались GPT-3.5/GPT-4 и собственные разбиения; фиксация строк и модели делает системы сопоставимыми, но меняет все абсолютные числа. Интерес представляет *относительное* поведение трёх конвейеров на одинаковом входе.

7.2. Результаты

Точность решателя и охват приведены в таблицах 2 и 3; числа Ankyra — это проверки из раздела 6.

Количество ложных доказательств: у Ankyra 0 всюду; у Logic-LM 2 (уровень A), 3 (уровень D), 0 (ProntoQA), 0 $\rightarrow$ 9 с self-refinement (FOLIO), 2 (AR-LSAT); у LINC 6 (уровень A), 11* (уровень D), 14 (FOLIO). Ноль у Ankyra структурен (раздел 7.4); ноль у внешней системы означает лишь, что исполнившееся подмножество случайно оказалось верным.

Стадия	Коллекция (n)	Ankyra	Logic-LM	+refine	LINC
L0	ProofWriter, уровень A (45)	100,0	8,9	—	86,7
L0	ProofWriter, уровень D (300)	99,0	6,7	—	28,3*
L1	ProntoQA, уровень a (48)	100,0	41,7	—	—
L2	FOLIO L2+отрицание (58)	69,0	6,9	39,7	74,1
L3	AR-LSAT, eval (30)	76,7	53,3	63,3	—

Таблица 2: Сравнение на общем входе: точность решателя в процентах от всех строк (неисполнимые программы и воздержания считаются ошибкой). LINC запускается в своём протоколе, кроме *уровня D ProofWriter, который запускался жадно (её значение с одним сэмплом на уровне A — 22,2%). «—» означает, что система не поддерживает коллекцию. Столбец +refine у Logic-LM — её self-refinement там, где он реализован в выпущенном коде (FOLIO, AR-LSAT).

Коллекция (n)	Ankyra	Logic-LM	+refine	LINC
ProofWriter, уровень A (45)	100	13,3	—	100
ProofWriter, уровень D (300)	100	7,7	—	32,0*
ProntoQA, уровень a (48)	100	41,7	—	—
FOLIO L2+отрицание (58)	98,3	6,9	55,2	98,3
AR-LSAT, eval (30)	100	60,0	70,0	—

Таблица 3: Охват: доля строк, программу которых символический решатель исполнил.

Первое впечатление обманчиво, и два факта это подтверждают. Во-первых, LINC достаточно конкурентна: на FOLIO она достигает 74,1% — *выше* 69,0% у Ankyra. Во-вторых, низкое число Logic-LM для этой модели реально: даже после двух запусков её собственного self-refinement она достигает лишь 39,7% на FOLIO и 63,3% на AR-LSAT, а для её наборов на PyKE такого механизма нет.

7.3. Перевод, а не рассуждение

Символические решатели — не узкое место. Если подать им написанные вручную формализации тех же 58 строк FOLIO, без модели в цикле:

Система	Решатель	Эталонный вход: точность	на исполнимых
LINC	NLTK/Prover9	54/58 (93,1%)	54/57 (94,7%)
Logic-LM	Prover9	47/58 (81,0%)	47/50 (94,0%)

Запуск решателя Logic-LM на её собственных сохранённых программах от GPT-4 также воспроизводит офлайн уровень статьи (точность на исполнимых: FOLIO 80,4%, ProofWriter 79,6%, ProntoQA 83,2%, AR-LSAT 60,0%). Решатели работают. Ломается интерфейс: Logic-LM и LINC просят модель написать программу текстом, а затем разбирают этот текст парсером. Ankyra просит структурированный JSON через tool calling и получает его без разбора. Повторный запуск сохранённых программ без модели показывает, где именно происходит сбой:

Коллекция (n)	Исполнено	Ошибка исп.	Провал парс.	Списки	голый х
ProofWriter, D (300)	23	214	63	276	179
FOLIO (58)	4	0	54	50	55
ProntoQA, a (48)	20	21	7	28	—
AR-LSAT (30)	18	12	0	0	0

Основные ошибки — поверхностные синтаксические детали, которых модель не воспро-

извела: маркдаун-списки, полужирный текст и голый x там, где грамматика Logic-LM требует $\$x$. AR-LSAT использует грамматику z3-выражений, которую модель в основном воспроизводит; её двенадцать отказов — дефекты собственного транслятора Logic-LM для z3 (увеличение таймаута исполнения с 1 с до 10 с ничего не меняет).

7.4. Разница — в интерфейсе

Требования к ответу модели у трёх систем разные. Ankyra извлекает данные в JSON-схему через tool/function calling, а детерминированный алгоритм превращает эту структуру в теорию; соблюдение формата обеспечивается контрактом API, а не подсказкой, поэтому модели нужно правильно передать лишь *содержание*. Logic-LM требует программу в написанной вручную грамматике (Predicates:/Facts:/Rules:/Query:, :::, $\$x$), которую читает чувствительный к синтаксису парсер. LINC требует каркас TEXT:/FOL: и исполняет извлечённые формулы; каркас достаточно прост, чтобы несколько из десяти сэмплов обычно оказались валидными, — поэтому её сэмплирование так важно (раздел 7.6). Центральный вывод: при малой и недорогой модели связывающим ограничением оказывается *интерфейс перевода*, а не рассуждение: Ankyra почти не чувствительна к особенностям форматирования, которые выбирает модель, конвейеры на свободном тексте — легко могут ломаться.

7.5. Чувствительность к модели

Чтобы понять, сколько в результате даёт сама LLM, а сколько — устройство конвейера, мы сравнили собственный тестовый набор ProofWriter из LINC на её же коде и протоколе на фиксированной подвыборке из 60 строк (каждая шестая строка). Наш запуск — только столбец DeepSeek. Значения GPT-4 и GPT-3.5 *не* являются запусками этой работы: они пересчитаны из генераций, закоммиченных авторами LINC в их репозиторий, конфигурация которых (8 примеров, десять сэмплов, температура 0,8) совпадает с использованным нами протоколом, а эталонные метки совпадают построчно с нашими. Сравнение, таким образом, — это сравнение режимов моделей на одной задаче и конфигурации, а не три запуска, выполненных здесь.

Модель (одна задача, код и конфигурация)	Точность
GPT-4 (сохранённые генерации LINC)	100,0%
GPT-3.5-turbo-16k (сохранённые генерации LINC)	96,7%
DeepSeek V4 Flash (наш запуск)	85,0%

Одна модель даёт разницу около двенадцати пунктов, а опубликованные 96–98% LINC на полном тесте согласуются со строками GPT. Опубликованные числа внешних систем опираются, таким образом, на GPT-3.5/GPT-4; вклад Ankyra здесь в том, что она сохраняет качество и на более слабой модели.

7.6. Контрольные прогоны декодирования

LINC: один прогон против родного протокола. На тех же строках и задачах переход LINC к её родному протоколу поднимает FOLIO с 34,5% до 74,1%, а ProofWriter уровня A — с 22,2% до 86,7%. Устойчивость LINC в существенной мере обеспечивается её шагом сэмплирования и голосования; приводить только жадное значение было бы введением в заблуждение.

Logic-LM: best-of- N . Logic-LM не сэмплирует. Как контроль, пять сэмплов на строку при температуре 0,8 на ProofWriter уровня A дали *ноль* исполнимых программ из 225 (жад-

ный запуск дал 6 из 45). Более высокая температура ухудшает соблюдение её грамматики, а не улучшает его, поэтому сэмплирование не объясняет разного поведения двух систем.

Logic-LM: self-refinement. Её собственный refinement, реализованный только для решателей Prover9 и z3, примерно утраивает охват FOLIO (с 6,9% до 55,2%), поднимает точность решателя на FOLIO с 6,9% до 39,7% и на AR-LSAT с 53,3% до 63,3%, но и вводит уверенно неверные ответы на FOLIO (с 0 до 9).

7.7. Что не запускается и почему

Сравнение ограничено кодом оцениваемых систем, а не основано на произвольном выборе. Реестр задач LINC содержит только FOLIO и ProofWriter, поэтому её нельзя запустить на ProntoQA, AR-LSAT или GSM8K; у Logic-LM нет возможности запуска на GSM8K. Self-refinement Logic-LM существует только для Prover9 и z3 — в каталоге её подсказок лежат лишь `self-correct-AR-LSAT.txt` и `self-correct-FOLIO.txt`, а список исполнителей перечисляет только эти два решателя, — поэтому для её наборов на PyKE (ProofWriter, ProntoQA) запускать нечего. Способность каждого конкретного решателя всё же измерена — эталонными прогонами выше, намеренно и без модели. Стадия L4 (арифметика) и ProntoQA-OOD не имеют аналога ни в одной из систем и остаются вне области.

7.8. Оговорки

Числа Ankuга взяты из зафиксированных проверок и реальных трасс запусков, но не перезапускались в тот же день, что и внешние системы; одна трасса уровня A ProofWriter — отказ извлечения с пустым вопросом (изменчивость поставщика), поэтому сохранённые трассы дают 44/45 против принятой проверки 45/45. Уровень D у LINC — жадный, а не родной, то есть нижняя оценка. Недетерминизм поставщика сохраняется даже при температуре 0; приводятся однократные запуски, как и везде в этой работе. Наконец, сравнение сквозное — подсказка, декодирование, парсер и решатель вместе, — это все дает правильное представление о том, насколько хорошо метод превращает малую модель в корректный ответ.

8. Ограничения

Размер выборки и статистика. Каждое число на реальной коллекции опирается на малую выборку ($N = 13, 30, 40, 42, 44, 300$), поэтому доверительные интервалы широки (таблица 1); на малых срезах они перекрываются с правдоподобными базовыми уровнями и не позволяют претендовать на передовой уровень. Чтобы обнаружить разницу около пяти процентных пунктов при $p \approx 0,95$ с мощностью 0,8, нужно порядка 400 примеров на группу.

Однократность запусков и недетерминизм. Реальный путь обращается к языковой модели, поставщик которой недетерминирован даже при нулевой температуре. Запуски выполняются один раз (по бюджетным соображениям) и повторяются лишь при расхождении, поэтому мы не приводим оценку дисперсии и не выполняем проверку значимости.

Подвыборки, а не полные тестовые выборки. Мы оцениваем зафиксированные подвыборки: ProntoQA-OOD, уровень a (42 из 5 800), AR-LSAT eval (30 из ~ 230), GSM8K eval

(40 из ~1 319), ProofWriter, уровень D (300 из 6 368 теорий). Полные коллекции не запускались, потому что бесплатного доступа к языковой модели нет, а запуски оплачиваются автором из собственных средств; это осознанная граница исследования, а не скрытый провал.

Корректность относительно формализации. Решатель корректен *относительно* теории, которую ему передали; перевод естественного языка в эту теорию выполняет языковая модель. Поэтому ошибка извлечения может дать *воздержание* или, в принципе, ответ `proven` о неверной формализации. Мы измеряем отсутствие уверенно неверных выводов (ноль на каждом отчитанном срезе) и снижаем риски извлечения проверкой дословности цитат, защитой от цитирования условных утверждений, защитой от опровержения на основе гипотез и контролем объявления предметной области. Это не доказательство истинности от начала до конца относительно реального мира.

У составной коллекции нет единого итогового показателя. Для FOLIO полный охват коллекции недостижим по построению: равенство, функциональные термы, исключаящее или, эквивалентность, кванторы по нескольким переменным и схемы аксиом лежат вне любой реализованной стадии, а полная выводимость в логике первого порядка лишь полуразрешима. Поэтому мы приводим стратифицированные срезы и явный счётчик `out_of_fragment`, а не оценку по всей коллекции.

Сравнение с опубликованными результатами. Раздел 7 действительно проводит сравнение на общем входе, но не с *опубликованными* числами: опубликованные значения получены на других коллекциях, разбиениях, протоколах и моделях, поэтому точности из разных работ по-прежнему не сравнимы напрямую. Сравнение — с выпущенными системами, перезапущенными на одинаковых входах и одной модели; вклад остаётся доводом о корректности, проверяемости и устойчивости к модели перевода, а точность — поддерживающим свидетельством.

Изменчивость модели и поставщика. Числа привязаны к поставщику извлечения на момент запуска; другая или обновлённая модель изменит столбцы текстового входа. Столбцы эталонного входа и синтетические проверки не зависят от модели.

Умолчания. Отчитана только синтетическая проверка; реальная коллекция отменяемых рассуждений (*defeasible*) и вывода на естественном языке (NLI) остаётся темой будущей работы.

9. Связанные работы

Карта области. Систематический обзор 2026 года [5] (178 работ, 2020–2025) выделяет четыре устойчивых паттерна нейросимволической интеграции: последовательный каскад, символический валидатор, символический планировщик с нейронным исполнителем и дифференцируемое объединение. Ankyra ближе всего к последовательному каскаду с ограниченным циклом, но отличается от него двумя вещами: протоколом предложений, при котором каждое утверждение входит в теорию только с цитатой или пометкой допущения, и объявленным фрагментом, при котором несовпадение требуемой и доступной логики даёт честный `out_of_fragment`, а не скрытый переход к более слабой процедуре. Тот же обзор отмечает, что мета-когнитивные способности — мониторинг собственных выводов, распознавание нехватки знаний, переключение режимов —

встречаются лишь в 5% работ; `out_of_fragment` и `proven_under(H)` можно рассматривать как вклад именно в эту нишу.

Ближайшая по духу система — *Autonomous Trustworthy Agents* (ATA) [6]: она также добивается детерминизма и проверяемости, но достигает этого *полным выносом* LLM за пределы runtime (LLM строит базу знаний офлайн, онлайн работает только символический решатель). Ankyra сохраняет LLM в цикле, ограничивая его протоколом; это принципиально разные точки на одном спектре. Тот же подход к верификации регуляторных документов через SMT-солвер описан в [7].

Рассуждения языковых моделей без сертификата. Цепочка рассуждений (chain-of-thought) [8] и самосогласованность (self-consistency) [9] показывают, что пошаговое порождение текста улучшает ответы языковых моделей. Однако такие цепочки не сертифицированы: шаг, выглядящий правдоподобно, может быть неверным, а у ответа нет ни истории вывода, ни различия «доказано» и «правдоподобно». Ankyra устраняет именно этот пробел: рассуждение порождает модель, но выводимость проверяет и удостоверяет символический решатель.

Языковая модель и символический решатель. Конвейеры вида «переводчик + решатель» (Logic-LM [3], LINC [4]) также передают рассуждение символическому решателю, но доверяют переводу целиком: если формализация неверна, неверно и заключение. Здесь граница строже: каждое утверждение входит в теорию только с дословной цитатой или явной пометкой допущения, обоснованность ответа выводится из его посылок (`proven` против `proven_under`), а несовпадение требуемой и доступной логики даёт честный `out_of_fragment`, а не скрытый переход к более слабой процедуре. Раздел 7 перезапускает обе системы на одинаковых входах и одной модели: их решатели корректны и состоятельны (эталонный вход даёт 81–93% на FOLIO), но как конвейеры на свободном тексте они чувствительны к оформлению вывода малой модели, тогда как структурированный интерфейс Ankyra — нет.

Коллекции как этапные проверки. Мы используем стандартные коллекции — ProofWriter [10], ProntoQA [11] и ProntoQA-OOD [12], FOLIO [13], AR-LSAT [14], GSM8K [15] — не как единый итоговый показатель, а как этапные проверки отдельных стадий, каждая со своей семантикой. Для составных коллекций (FOLIO) мы выделяем внутрифрагментные срезы и отдельно отчитываем `out_of_fragment`; единый балл по коллекции со смешанными конструкциями недостижим по построению.

Немонотонные рассуждения и умолчания. Логика умолчаний [16] и циркумскрипция [17] формализуют отменяемость выводов; Ankyra реализует слой умолчаний со специфичностью через `is_a` и честно сообщает о неразрешённом конфликте, не угадывая победителя. Философская рамка (doxa/logos) и семейство операторов взаимодействия изложены в сопутствующей работе [2]; здесь мы сосредоточены на системе, реализующей этот интерфейс.

История вывода и проверяемость. Опора на автоматически собранную историю вывода сближает Ankyra с работами о проверяемом выводе. Объяснение здесь — изложение доказательства, а не второе мнение о нём: модель не вправе добавлять в него факты, а каждый шаг называет сработавшее правило, использованные связывания и обосновывающую цитату или гипотезу.

10. Заключение

Мы представили нейросимволическую систему, в которой языковая модель предлагает, а детерминированный решатель решает, охватывающую семейство разрешимых формализмов в рамках протокола объявленного фрагмента. Вклад — не отдельный решатель, а архитектура и дисциплина: протокол предложений, при котором ни одно утверждение не входит в теорию без цитаты или пометки допущения; обоснованность ответа, никогда не превышающая обоснованности его посылок; объявленный фрагмент и честное `out_of_fragment` вместо молчаливого перехода к более слабой логике; и методика этапных проверок составных коллекций, разделяющая охват метода и потолок извлечения.

На зафиксированных срезах стандартных коллекций система показывает высокую точность, не допуская ни одного уверенно неверного вывода, и честно воздерживается там, где её логика не может выразить задачу. Сравнение на общем входе с Logic-LM и LINC показывает, что их символическое рассуждение корректно, но узким местом при малой модели оказывается интерфейс перевода на свободном тексте, тогда как структурированное извлечение Ankuга к нему устойчиво; при более сильной модели и родном сэмплировании LINC конкурентна. Оценка — демонстрационный прототип, а не передовой уровень; ограничения выше определяют границу того, что поддерживают эти результаты. Естественные направления дальнейшей работы — реальная коллекция отменяемых рассуждений (defeasible) и NLI, конечные функциональные термы и кванторы по нескольким переменным (сейчас за пределами фрагмента), а также запуски на полных коллекциях по мере появления бюджета.

А. Фрагменты и переключатели

Система выводит требуемый фрагмент из *построенных* теории и запроса; переключатели объявляют, какие процедуры запуску разрешены. Ни то, ни другое не задаёт модель.

Признак фрагмента	Когда возникает
horn	нет нехорновых голов, нет кванторов существования, цель одиночная
negation	непустые ограничения несовместности или отрицательный литерал
disjunction	дизъюнктивная голова или дизъюнктивный заземлённый факт
existential	непустые экзистенциальные утверждения теории или экзистенциальная цель
compound_goal	<code>goal_mode</code> $\neq$ <code>single</code> (конъюнктивная или дизъюнктивная цель)
builtin equality	предикат сравнения (<code>=</code> , <code>neq</code> , <code><</code> , <code>lte</code> , <code>></code> , <code>gte</code>) зарезервированный предикат <code>eq</code> в любом месте

Переключатель	По умолчанию	Действие
ANKYRA_MAX_WAVES	8	максимум итераций рассуждения
ANKYRA_ALLOW_HYPOTHESES	true	разрешены ли допущения
ANKYRA_NEGATION_MODE	open	мировое допущение по умолчанию для L1
ANKYRA_LOGIC	off	включить клаузальную процедуру L2
ANKYRA_DEFEASIBLE	off	включить слой умолчаний
ANKYRA_BUILTINS	false	включить слой сравнений
ANKYRA_CSP	off	включить конечно-доменный решатель L3
ANKYRA_ARITH	off	включить числовой решатель L4
ANKYRA_EXTRACT_SAMPLES	1	число попыток извлечения

В. Воспроизведение

Проверки без участия языковой модели (синтетические наборы и диагностика с эталонным входом) воспроизводимы из репозитория без доступа к языковой модели и без оплаты API:

```
uv run python -m evals.routing_synthetic
uv run python -m evals.l1_synthetic
uv run python -m evals.l2_synthetic
uv run python -m evals.l3_synthetic
uv run python -m evals.l4_synthetic
uv run python -m evals.defeasible_synthetic
uv run python -m evals.ar_lsats --gold evals/data/ar_lsats_gold.jsonl
uv run python -m evals.gsm8k --gold evals/data/gsm8k_gold.jsonl
uv run python -m evals.analyze_folio
```

Запуски с обращением к языковой модели (к платному поставщику извлечения) выполняются один раз и повторяются только при расхождении:

```
uv run python -m evals.proofwriter --tier d
uv run python -m evals.prontoqa --tier b
uv run python -m evals.prontoqa_ood --tier a
uv run python -m evals.ar_lsats --sample eval --live
uv run python -m evals.gsm8k --sample eval --live
```

Сравнение на общем входе из раздела 7 воспроизводимо из репозитория: адаптеры и драйверы лежат в `comparison/`, построчные выходы — в `comparison/results/`. Офлайн-части (адаптеры, проверки бэкендов на эталонном входе) не требуют API; реальные части — это записанные там платные запуски.

```
PYTHONPATH=$PWD /tmp/opencode/cmp/llm-venv/bin/python -m comparison.run_logicllm
--repo /tmp/opencode/cmp/logicllm --action prepare
PYTHONPATH=$PWD /tmp/opencode/cmp/llm-venv/bin/python -m comparison.run_logicllm
--repo /tmp/opencode/cmp/logicllm --action all --collection folio_tier_a
PYTHONPATH=$PWD /tmp/opencode/cmp/linc-venv/bin/python -m comparison.run_linc \
--repo /tmp/opencode/cmp/linc --collection folio_tier_a --nshot 8 \
--n-samples 10 --temperature 0.8 --jobs 6
```

Список литературы

- [1] Ankyra: A domain-agnostic neuro-symbolic reasoning engine. GitHub repository. URL: <https://github.com/merfill/ankyra>
- [2] Лапшин В. Докса и логос: формализация функций неформализуемого источника в нейросимволических системах. (сопутствующая работа, в подготовке).
- [3] Pan L. et al. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. Findings of EMNLP. 2023.
- [4] Olausson T. X. et al. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. EMNLP. 2023.
- [5] Bin Hakim S., Adil M., Velasquez A., Song H. H. Neuro-symbolic agentic AI: Architectures, integration patterns, applications, open challenges and future research directions // Computer Science Review. 2026. Vol. 60. Art. 100902. DOI: 10.1016/j.cosrev.2026.100902.
- [6] Peer D., Stabinger S. ATA: Autonomous Trustworthy Agents. arXiv:2510.16381.
- [7] Hsia Y.-S., Yu F., Jiang J.-H. R. Neuro-Symbolic Compliance: LLM-driven SMT verification of regulatory documents. arXiv:2601.06181.
- [8] Wei J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903.
- [9] Wang X. et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. arXiv:2203.11171.
- [10] Tafjord O., Mishra B. D., Clark P. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. AI2, arXiv:2012.13048.
- [11] Saparov A., He H. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. ICLR 2023. arXiv:2210.01240.
- [12] Saparov A., He H. Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples. NeurIPS 2023. arXiv:2305.15269.
- [13] Han S. et al. FOLIO: Natural Language Reasoning with First-Order Logic. arXiv:2209.00840, v3 2024.
- [14] Zhong W. et al. AR-LSAT: Investigating Analytical Reasoning of Text. 2021. arXiv:2104.06598.
- [15] Cobbe K. et al. Training Verifiers to Solve Math Word Problems. 2021. arXiv:2110.14168.
- [16] Reiter R. A Logic for Default Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 81–132.
- [17] McCarthy J. Circumscription—A Form of Non-Monotonic Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 27–39.